Drive Music — System Specification

Version: 0.1.0 **Stack:** Next.js 16 (App Router, Turbopack) · React 19 · TypeScript · Tailwind CSS v4 · NextAuth v4 · IndexedDB (`idb`) **Scope:** A single-user, client-heavy Progressive Web App that plays music streamed from the user's own Google Drive, caches it for offline listening, and personalizes shuffle/recommendations with a small on-device neural network.

---

1. Overview

Drive Music is a personal music player with no backend database and no server-side business logic beyond OAuth token exchange. Every feature — Drive browsing, offline caching, playlists, favorites, listening-behavior modeling, and recommendations — runs **entirely in the browser**, persisted in **IndexedDB**. The only network dependencies at runtime are the Google OAuth/Drive APIs.

Core design principles followed throughout the build:

- **No unnecessary dependencies.** The recommendation engine is a hand-written ~150-line neural net, not a machine-learning framework. The service worker is hand-written, not a webpack-based plugin (which Next's own docs flag as incompatible with this project's Turbopack build).
- **Client-only persistence.** IndexedDB is used the same way for songs, playlists, the listening model, and its training history — one database, one access pattern.
- **Graceful degradation.** An untrained recommendation model behaves like plain shuffle; a track with no embedded metadata still plays, just without cover art.

---

2. Tech Stack

Layer	Choice	Notes
Framework	Next.js 16.2.12 (App Router)	Turbopack for both `dev` and `build`
UI	React 19, Tailwind CSS v4	CSS-based Tailwind config (no `tailwind.config.js`); custom keyframes in `globals.css`
Auth	NextAuth v4 (`next-auth`)	Google provider only
Local storage	IndexedDB via `idb`	One database, six object stores (see §5)
Drive access	Google Drive API v3	Called directly from the browser with the session's access token
Metadata parsing	`music-metadata` (browser build)	Reads ID3v2 / MP4 / FLAC / Vorbis tags directly from downloaded blobs
Icons	`lucide-react`	
Utility	`clsx`	Conditional class names

---

3. Architecture & Provider Hierarchy

The app is a single root layout (`src/app/layout.tsx`) wrapping every route in `Providers` (NextAuth's `SessionProvider` , plus service-worker registration). The main experience lives entirely under `/` (`src/app/page.tsx`), gated on `useSession()` :

```
RootLayout
└─ Providers (SessionProvider, registers /sw.js)
   └─ page.tsx  → SignInScreen (signed out) | MusicApp (signed in)
                     └─ PlayerProvider        (playback, cache, shuffle, the AI model)
                        └─ PlaylistsProvider  (playlists — favorites are one of these)
                           └─ AppShell        (tab chrome: Home / Browse / Playlists /
Library)
                              ├─ HomeView | DriveBrowser | PlaylistsView/Detail |
LibraryView
                              ├─ Player       (persistent mini bar)
                              └─ FullPlayer   (expandable "Now Playing" overlay)
```

`/admin` is a **separate route**, outside this provider tree — it reads directly from the IndexedDB helpers in `src/lib/db.ts` rather than through React context, since it only needs read-only snapshots for display, not live playback state.

---

4. Authentication

- **Provider:** Google, via `next-auth` 's `GoogleProvider` (`src/lib/auth.ts`).
- **Scope requested:** `openid email profile https://www.googleapis.com/auth/drive.readonly` .
- **Consent parameters:** `access_type=offline&prompt=consent` , so a refresh token is issued on first sign-in (required for long-lived access beyond the ~1 hour access-token lifetime).
- **Token refresh:** the NextAuth `jwt` callback checks `accessTokenExpires` ; when expired, it POSTs to `https://oauth2.googleapis.com/token` with the stored refresh token and re-issues both tokens. Failure sets `token.error = "RefreshAccessTokenError"` , surfaced in the UI as a "sign in again" banner.
- **Session shape:** the session's `accessToken` is exposed to client components (module augmentation in `src/types/next-auth.d.ts`) so the browser can call the Drive API directly — deliberate, since the browser is what needs to stream/cache the actual audio bytes; proxying every byte through a Next.js server would add no security benefit for a single-user app.

---

5. IndexedDB Data Model

Database name: `drive-music` (current version: **6**). All access goes through `src/lib/db.ts` , which owns the single `openDB` connection and every store's CRUD.

Store	Key	Value shape	Purpose
`tracks`	`fileId`	`CachedTrack` — `{ fileId, blob, mimeType, driveMeta, parsedMeta, cachedAt }`	The offline cache. Storing the raw audio `Blob` here is what makes playback work with no network once cached.
`playlists`	`id`	`Playlist` — `{ id, name, tracks: DriveFile[],`	User-created playlists **and** the auto-managed `"Favorites"` playlist (see §9).

		`createdAt }`	Stores full `DriveFile` snapshots so a playlist can list tracks that haven't been downloaded yet.
`recentSources`	`id`	`RecentSource` — `{ type: "folder" | "playlist" | "library", id, name, tracks, lastPlayedAt }`	Backs Home's "Recently played" row. Upserted every time a queue is started with an attached `PlaySource`.
`model`	`"default"` (singleton)	`ListeningModel` — see §7	The on-device recommendation model's weights.
`modelEvents`	`id`	`ModelEvent` — `{ id, trackId, title, fraction, predicted, at }`	A capped (500-row) training log: what the model predicted for a track *before* training on it, versus what actually happened. Powers the Admin Dashboard's history table.

`DriveFile` (`src/types/index.ts`) is the canonical "a file as Drive describes it" shape (`id, name, mimeType, size?, modifiedTime?, thumbnailLink?, iconLink?`), used everywhere a track is referenced before/without being cached. `ParsedMetadata` (`title?, artist?, album?, year?, durationSec?, pictureDataUrl?`) is the richer, ID3-derived shape attached once a track is cached.

---

6. Google Drive Integration (`src/lib/drive.ts`)

- **Listing a folder:** `GET /drive/v3/files` with query `'<folderId>' in parents and trashed = false and (mimeType = 'application/vnd.google-apps.folder' or mimeType contains 'audio/')` , `orderBy=folder,name` , paginated via `nextPageToken` . The root folder is Drive's special `'root'` id; `DriveBrowser.tsx` keeps a breadcrumb stack of `{id, name}` as the user navigates.
- **Downloading a file:** `GET /drive/v3/files/{fileId}?alt=media` with `Authorization: Bearer <accessToken>` , read as an `ArrayBuffer` and wrapped in a `Blob` tagged with the file's own `mimeType` .
- All calls are plain `fetch` from client components — no server-side Drive proxy exists.

---

7. The On-Device Recommendation Model

7.1 What it is

A small, **online-trained 2-layer neural network** (`src/lib/model.ts`): 47 input features → 12-unit `tanh` hidden layer → 1 sigmoid output. It predicts, for a given track and moment in time, *"what fraction of this track will you actually listen to"* — a continuous score in `[0, 1]` used both as a **shuffle weight** and a **recommendation score**. It is not a heuristic or a hand-tuned rule — its weights are learned by gradient descent from real listening behavior, just at a scale that needs no ML framework and trains instantly in the browser.

7.2 Features (`src/lib/features.ts` , `FEATURE_SIZE = 47`)

All features are one-hot within fixed-size hashed buckets, so vocabulary size (arbitrary artist/album/track names) never changes the input dimension:

Group	Size	Encoding
Bias	1	Always 1
Time of day	4	One-hot: night (0–6h) / morning / afternoon / evening
Weekday vs. weekend	2	One-hot
Artist	16	String hashed into 16 buckets
Album	16	String hashed into 16 buckets
Track id	8	Drive file id hashed into 8 buckets — lets the model learn per-track affinity even without metadata

`extractFeatures(file, parsedMeta, at: Date)` builds this vector; it degrades gracefully to "unknown-artist"/"unknown-album" buckets plus the track-id hash when a file isn't cached yet (so uncached tracks can still be scored/shuffled sensibly).

7.3 Architecture & training (`src/lib/model.ts`)

```
input(47) → w1[12×47] + b1[12] → tanh → hidden(12) → w2[12] + b2 → sigmoid → output(1)
```

- **Init:** hidden layer (`w1`) starts with small random weights (`±0.1`) to break symmetry between hidden units; output layer (`w2` , `b2`) starts at zero.
- **`predict(model, features)` :** one forward pass, returns the sigmoid output.
- **`trainStep(model, features, label)` :** one online backprop step — output error `label − prediction` , standard sigmoid/cross-entropy delta rule at the output layer, propagated through `tanh` 's derivative (`1 − tanh²`) to the hidden layer, fixed learning rate `0.05` . Returns a new model with `trainingEvents` incremented.

7.4 The training signal

Nothing about training requires dedicated UI — it happens automatically on every track transition, in `PlayerContext.tsx` 's track-load effect:

1. Just before swapping the `<audio>` element's `src` to a new track, if a previous track was playing, its **actual fraction played** is read straight from the DOM (`audio.currentTime / audio.duration` , clipped to `[0, 1]`) — reading the DOM directly (not React state) avoids any state-timing races.
2. `extractFeatures` is built for that **outgoing** track using whatever metadata was current at that point in the closure.
3. `predict()` is called first (for logging — see `ModelEvent`), then `trainStep()` updates the model.
4. Both the updated model (`saveModel`) and a `ModelEvent` log row (`recordModelEvent`) are persisted.

This means a track played to completion trains toward label `≈1` ; an early skip trains toward whatever fraction was actually heard (e.g. `0.1` for a fast skip) — a richer signal than a binary like/skip.

A `modelInitializedRef` guard ensures the model isn't overwritten with fresh zero-weights before its saved state has finished loading from IndexedDB on mount.

7.5 Where the model is used

- **Weighted shuffle** (`PlayerContext.tsx`): see §8.3.
- **Home → "Made For You"** (`HomeView.tsx`): every cached track is scored with `predict()` against the *current* moment (so recommendations can shift by time of day), sorted descending, top 20 become a playable mix. Its subtitle reads "Learning your taste…" below 5 training events, "Based on your listening" after.
- **"Shuffle play" starting track** (`LibraryView.tsx` , `CollectionDetail.tsx`): the very first track chosen when hitting "shuffle" is a **weighted random pick** (`weightedRandomIndex` , roulette-wheel sampling over `predict()` scores), not a uniform random index — every random choice in the app is model-influenced, none fall back to plain `Math.random() * length` .

---

8. Playback System (`src/components/PlayerContext.tsx`)

This is the largest and most stateful module — a single context providing the current queue, the `<audio>` element, and every transport/queue action.

8.1 Core state

`queue: DriveFile[]` , `currentIndex` , `currentMeta` (parsed metadata of the current track), `isPlaying` , `isLoading` , `error` , `progress` / `duration` (mirrors of the `<audio>` element), `volume` , `shuffle` , `loopMode: "off" | "all" | "one"` , `isExpanded` (full-player visibility), `cachedTracks` (a live `Map<fileId, CachedTrack>`), `recentSources` , `model` , `shuffleOrder` (see §8.3).

8.2 Loading a track

A single `useEffect` keyed on `currentFile?.id` : trains the model on the *outgoing* track (§7.4) → resolves the *incoming* track via `ensureCached()` (IndexedDB hit, or `downloadFile` + `parseTrackMetadata` + `putCachedTrack` on miss) → creates an object URL from the blob and assigns it to the `<audio>` element → plays. This same function is what "download for offline" *is* — the first play of any track is also its cache-write.

8.3 Shuffle — a weighted shuffle-bag, not uniform random

Classic "true random" shuffle can replay a track soon after it played and can skip others entirely — this app instead deals a **weighted permutation** of the whole queue (Efraimidis–Spirakis weighted sampling: `key = random() ** (1/weight)` , sorted descending, current track pinned first) so:

- Every track plays exactly once before any repeat (still a full permutation).
- Higher model-predicted tracks tend to sort earlier within that pass.
- An untrained model gives every track equal weight, so it degrades to plain uniform shuffle with no special-casing needed.

`shuffleOrder` is **reactive state** (not a ref) specifically so the "Up Next" list in `FullPlayer.tsx` can display the true upcoming order — it's recomputed eagerly the moment shuffle is toggled on, or a new queue starts, not lazily on the next skip. `next()` / `prev()` /track-end each resolve-or-regenerate this order and advance a derived position (`shuffleOrder.indexOf(currentIndex)`) rather than tracking a separate position counter, so it can never drift out of sync with the queue.

Loop-mode interaction: `loopMode: "one"` always replays the current track regardless of shuffle; manual skip (prev/next buttons) always advances/wraps regardless of loop mode; only the *natural end of a track* respects `loopMode: "off"` by stopping instead of wrapping (sequential) or reshuffling (shuffled).

8.4 Other queue actions

- **`play(queue, index, source?)`** — replaces the whole queue and jumps to `index`; if `source` (a `PlaySource`) is given, upserts a `RecentSource` for Home's "Recently played".
- **`addToQueue(file)`** — appends to the end of the current queue without interrupting playback (starts playing immediately only if nothing was queued).
- **`downloadAll(files)`** — sequentially caches every not-yet-cached file in a list, exposing `{ done, total }` progress for a progress UI.
- **`removeFromCache(fileId)`** — deletes a cached blob.

---

9. Playlists & Favorites (`src/components/PlaylistsContext.tsx`)

Playlists are a flat list of `{ id, name, tracks: DriveFile[], createdAt }`, CRUD'd through `src/lib/db.ts`. **Favorites are not a separate feature** — "favoriting" a track adds it to (and removes it from) a lazily-created, ordinary playlist named `"Favorites"` (`FAVORITES_PLAYLIST_NAME`). `isFavorite(fileId)` and `toggleFavorite(file)` are just membership checks/mutations against that one playlist:

```
toggleFavorite(file):
  playlist = playlists.find(name === "Favorites") ?? createPlaylist("Favorites")
  if file already in playlist.tracks → removeTrackFromPlaylist
  else                               → addTrackToPlaylist
```

Because it's a real playlist, it automatically appears in the Playlists tab and in Home's "Your playlists" row (sorted first, with a heart icon in place of cover art when no covers are cached) — no separate storage, no separate UI surface to maintain. The heart toggle is available everywhere a track is rendered: Browse, Library, Playlists, Up Next, and the main Now Playing view (`TrackRow.tsx`, `FullPlayer.tsx`).

---

10. Pages & Key Components

View	File	Responsibility
Sign-in	`SignInScreen.tsx`	Minimal centered "Sign in with Google" card
Home	`HomeView.tsx`	Greeting; "Recently played", "Your playlists", "Recommended for you" (Shuffle All / Recently Added / Made For You) rows of `PlaylistCard`s; clicking a card opens `CollectionDetail` rather than auto-playing
Collection preview	`CollectionDetail.tsx`	Generic "preview and play" view for any track list (a recent source, a playlist, an auto-mix) — explicit Play-in-order / Shuffle buttons, download-all, and the full track list so a specific track can be picked directly
Browse	`DriveBrowser.tsx`	Breadcrumb Drive folder navigation, folder/audio-file listing, download-all for the current folder
Playlists	`PlaylistsView.tsx` / `PlaylistDetail.tsx`	Create/delete/rename playlists; view and play one

Library	`LibraryView.tsx`	All cached (offline-available) tracks; "Shuffle play" (model-weighted starting track)
Mini player	`Player.tsx`	Persistent bottom bar: cover, title/artist, shuffle/repeat, transport, seek, volume, expand
Now Playing	`FullPlayer.tsx`	Full-screen overlay: large cover art with an ambient glow (average color sampled from the artwork via an offscreen canvas, `src/lib/color.ts`), favorite toggle, transport, volume (mute toggle + dynamic icon), and a redesigned "Up Next" card listing the true shuffle-aware upcoming queue via `TrackRow`. Stays mounted and slides/fades via CSS transitions rather than mounting/unmounting, so expand/collapse animates.
Admin Dashboard	`src/app/admin/page.tsx`	Usage stats (cache size, playlist/track counts, browser storage quota), model internals (architecture, weight norm/min/max, a weight-magnitude-by-feature-group bar chart), a recent-training-events table, and a "Download model" button that exports the current weights as JSON
Track row	`TrackRow.tsx`	The single shared row renderer used by every list above — cover, cached badge, favorite, add-to-queue, add-to-playlist, remove (context-dependent)

---

11. Progressive Web App

- **Manifest** (`src/app/manifest.ts` , Next's native file convention → served at `/manifest.webmanifest`): name, `display: "standalone"` , theme/background color, 192×192 and 512×512 icons (including a `maskable` variant).
- **Icons** are generated at build time with `next/og` 's `ImageResponse` (`src/app/icon.tsx` , `icon-192.png/route.tsx` , `icon-512.png/route.tsx`) — no external image tool or binary asset needed, and each route is marked `force-static` so it's rendered once at build time, not per-request.
- **Service worker** (`public/sw.js`), registered from `Providers.tsx` : a small hand-written stale-while-revalidate cache for same-origin `GET` requests, **explicitly excluding `/api/*`** so auth/session endpoints always hit the network fresh (a cached stale session response would be a real correctness bug). Deliberately not using a webpack-based generator plugin (e.g. `next-pwa` /Serwist) — Next's own PWA guide notes those require webpack configuration that a Turbopack build (this project's default) won't apply.
- **iOS/metadata:** `appleWebApp` (capable, status bar style, title) in `layout.tsx` 's metadata, plus a `viewport` export with `themeColor` .
- Note that offline *audio playback* needs no service worker at all — it's served from IndexedDB blobs. The service worker's job is purely to let the app shell (HTML/JS/CSS) load with no connectivity after the first visit.

---

12. Known Limitations / Non-Goals

- **Single user, no server-side state.** There is no multi-user concept anywhere — "Admin Dashboard" is a personal stats page, not a multi-tenant admin panel.

- **The model is intentionally small.** A 12-unit hidden layer trained on one person's casual listening will not rival a production recommender — it's meant to feel like a personal touch, not a music-discovery engine.
- **"Up Next" reflects `queue` order when shuffle is off**, and the true `shuffleOrder` when shuffle is on; it does not currently support manual drag-to-reorder.
- **No offline access to the Drive file *listing*** — Browse requires network to list folders; only already-cached tracks play offline.
- **Google OAuth app is unverified/in "Testing"** — only accounts added as test users in Google Cloud Console can sign in until/unless the OAuth consent screen is submitted for verification.
- **No push notifications** — the PWA guide's push-notification path (VAPID keys, `web-push`, Server Actions) was not implemented; out of scope for what was asked.